

Módulo 6

Estruturas de Dados Dinâmicas

Ponteiros

Para declararmos um ponteiro, basta utilizar o operador ***(asterisco)** antes do nome da variável.

Ponteiros são tipados, ou seja, devem ter seu tipo declarado e somente podem apontar para variáveis do mesmo tipo.

```
int main() {  
    int *numPtr;  
    return 0;  
}
```

Operadores para Ponteiros

- &** - Fornece o endereço de memória onde está armazenado uma variável. Lê-se **“O endereço de”**.
- *** - Valor armazenado na variável referenciada por um ponteiro. Lê-se **“O valor apontado por”**.

Ponteiros

Os ponteiros são variáveis cujo conteúdo é um endereço de memória.

Assim, um ponteiro endereça uma posição de memória que contém valores que são na verdade endereços para outras posições de memória.

```
int main() {  
    int num = 5;  
    char ch = 'x';  
    int *numPtr = &num;  
    return 0;  
}
```

Endereço	Conteúdo	Identificador
0x0100	5	num
0x0101		
0x0102		
0x0104	'x'	ch
0x0105	0x0100	numPtr



Exercício

O que será impresso no ecrã?

```
int main() {  
    int num = 5;  
    char ch = 'x';  
    int *numPtr = &num;  
  
    printf("%d\n", *numPtr);  
    printf("%p\n", numPtr);  
    printf("%p\n", &numPtr);  
  
    return 0;  
}
```

Resolução:

- 5
- 0x0100
- 0x0105

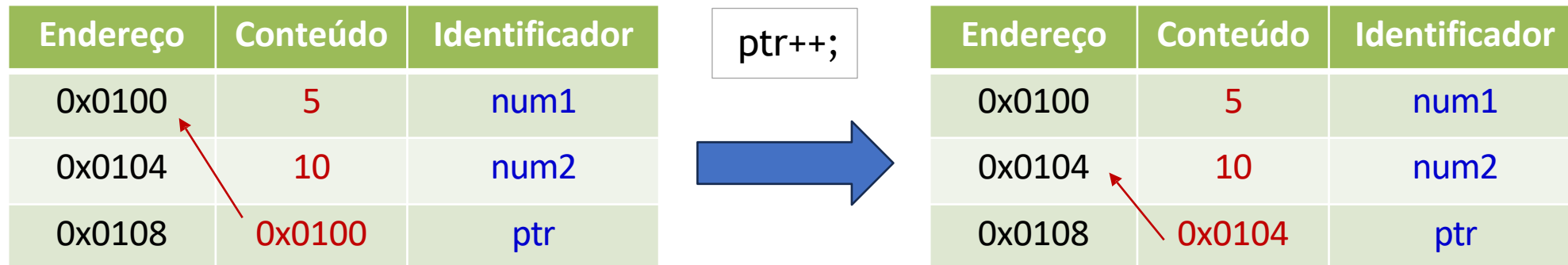
Ponteiros

Repare que estamos fazendo com que p1 aponte para o mesmo endereço que o p2.

```
int main() {  
    int num = 5;  
    int *P1 = &num;  
    int *P2;  
    P2=P1;  
    return 0;  
}
```

Aritmética de Apontadores

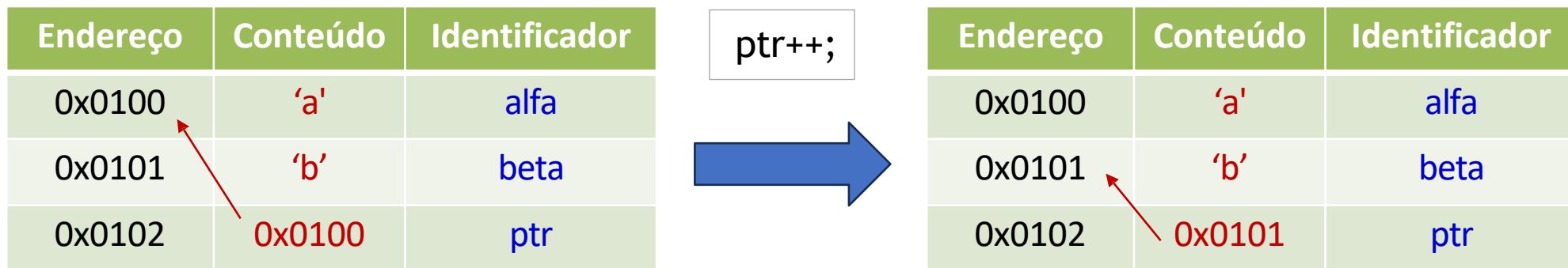
É possível somar ou subtrair um valor inteiro a um apontador?



Aritmética de Apontadores

O número de bytes que o apontador avança/retrocede depende do seu tipo:

char: 1 byte; int: 4 bytes; double: 8 bytes;



Complete o código

```
int main() {  
    int num = 55, valor = 0;  
    int *p;  
    p = ; /* Complete aqui! */  
    valor = ; /* Complete aqui! */  
  
    printf ("%d\n", valor);  
    printf ("Endereco para onde o ponteiro aponta: %p\n", p);  
    printf ("Valor da variavel apontada: %d\n", *p);  
    return 0;  
}
```

Resolução

```
int main() {  
    int num = 55, valor = 0;  
    int *p;  
    p = &num ; /* Complete aqui! */  
    valor = *p ; /* Complete aqui! */  
  
    printf ("%d\n", valor);  
    printf ("Endereco para onde o ponteiro aponta: %p\n", p);  
    printf ("Valor da variavel apontada: %d\n", *p);  
    return 0;  
}
```

Apontadores e Vetores

Apontadores e vetores

Um vetor em C é na verdade um apontador para a primeira posição de memória onde está guardado o vector;

```
int main() {  
    char texto[128];  
    scanf("%s", texto);  
    return 0;  
}
```

Endereço	Conteúdo	Identificador
0x0100	0x2000	texto
...		
0x2000	O	texto[0]
0x2001	L	texto[1]
0x2002	Á	texto[2]
0x2003		texto[3]
...		

Apontadores e vetores

Um vector em C é na verdade um apontador para a primeira posição de memória onde está guardado o vector;

```
int main() {  
    int valores [10] = {3, 5, 8, 3};  
    int *ptr;  
    ptr = valores;  
    return 0;  
}
```



Exercício

Encontre o erro no código a seguir:

```
int main() {  
    int a[10];  
    int *p;  
  
    p = a;  
    a = p;  
    p++;  
    a++;  
    return 0;  
}
```

Resolução:

- `a = p;` // ERRO
- `a++` // ERRO

Desafio

Crie um programa em C que utilize ponteiros para manipular vetores. O programa deve declarar um vetor de inteiros com um tamanho definido pelo utilizador. Peça ao utilizador para preencher o vetor com valores à sua escolha. Em seguida, o programa deve encontrar e exibir o maior e o menor valor presente no vetor, também através do uso de ponteiros. Para finalizar, o programa deve imprimir todos os elementos do vetor, utilizando novamente um ponteiro.

Apontadores e Funções

Apontadores e funções

Quando se passa um vetor como parâmetro para uma função, o que é passado é o endereço do primeiro elemento do vetor.

```
int somar(int numeros[tamanho]);
```

```
int somar(int numeros[]);
```

```
int somar(int *numeros);
```

Apontadores e funções

main.c

```
int main(){
    int notas[10] = {1,2,0,7,7,7,2,4,16,8};
    int numNeg = negativas(notas, 10);

    printf("numero de negativas: %d\n", numNeg);
    return 0;
}
```

funções.c

```
int negativas (int *notas, int tam){
    int neg = 0;
    for (int i = 0; i < tam; i++, notas++){
        if (*notas < 9.5){
            neg++;
        }
    }
    return neg;
}
```

Apontadores e funções

main.c

```
int main(){  
    int notas[10] = {1,2,0,7,7,7,2,4,16,8};  
    int numNeg = negativas(notas, 10);  
  
    printf("numero de negativas: %d\n", numNeg);  
    return 0;  
}
```

funções.c

```
int negativas (int *notas, int tam){  
    int neg = 0;  
    for (int i = 0; i < tam; i++){  
        if (*(notas + i) < 9.5){  
            neg++;  
        }  
    }  
    return neg;  
}
```

Apontadores e funções

main.c

```
int main(){  
    int notas[10] = {1,2,0,7,7,7,2,4,16,8};  
    int numNeg = negativas(notas, 10);  
  
    printf("numero de negativas: %d\n", numNeg);  
    return 0;  
}
```

funções.c

```
int negativas (int *notas, int tam){  
    int neg = 0;  
    for (int i = 0; i < tam; i++){  
        if (notas[i] < 9.5){  
            neg++;  
        }  
    }  
    return neg;  
}
```

Desafio

Crie um programa em C que utiliza ponteiros para manipular um array de inteiros. O programa deve ter a seguinte funcionalidade:

A função *duplicarArray()* recebe um ponteiro para um array de inteiros e o número de elementos. Esta função deve criar um novo array que contém os mesmos valores que o array original, mas duplicados. Por exemplo, se o array original é [1, 2, 3], o novo array deve ser [1, 1, 2, 2, 3, 3]. A função deve também imprimir o novo array.